

SIMATS ENGINEERING

ASSESSMENT TOOL 2

**COURSE CODE: CSA14
DESIGN**

COURSE NAME: COMPILER

COURSE OUTCOME COVERED

CO3: Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)

Assessment Tool Weightage: 20%

QUESTIONS:

Total Marks:100

Annexure A

Debugging

Code of Conduct:

*I __K.Madhulika(192524190)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:Madhulika

Q. No	Understanding of Debugging Concepts (25)	Error Identification(25)	Root Causes Analysis (20)	Error Corrections and Solution Design(20)	Testing and Validation(10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 25	/ 20	/ 20	/ 10	/ 100

Annexure A

Debugging

1. A syntax directed definition for the expression $a + b * c$ generates an incorrect annotated parse tree because the semantic action for $+$ is evaluated before the attributes of $b * c$ are computed. Identify the error in the semantic rules and modify the rules so that the attributes are evaluated according to the correct dependency order and operator precedence.
2. A compiler constructs an incorrect syntax tree for the expression $a * (b + c) - d$, where the $+$ node is incorrectly placed as a child of the $-$ node and the multiplication relationship is lost. Trace the tree construction during parsing, identify the faulty semantic action, and provide the corrected syntax tree construction rules.
3. A bottom up parser uses an S attributed definition to evaluate the expression $x = (a + b) * c$, but the synthesized attribute of the multiplication node is computed before the addition node is reduced. Identify the incorrect reduction or semantic action and rewrite the rules so that the syntax tree and synthesized attributes are correctly evaluated during bottom up parsing.
4. An L attributed definition is used to translate the declaration `int a, b, c`, but during top down parsing the declared type `int` is assigned only to `a` and is not propagated to `b` and `c`. Identify the error in the inherited attribute propagation and modify the semantic actions so that the type information is correctly passed to every identifier.
5. A bottom up translation scheme for the expression $a + b * c$ produces intermediate results in the order $t1 = a + b$ and $t2 = t1 * c$. Identify the error in the attribute evaluation or semantic actions that causes the incorrect evaluation order. Modify the translation scheme so that multiplication is evaluated before addition while maintaining correct bottom up attribute evaluation.

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Debugging Concepts	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Error Identification	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Root Causes Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Error Correction & Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Testing and Validation	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Debugging

CSA 1419 - Compiler Design

Name: K. Madhulika

Reg no: 192524190

1. Error:

The incorrect syntax-directed definition is

$$E \rightarrow E_1 + T \quad E.val = E_1.val + T.val$$

$$T \rightarrow T_1 * F \quad T.val = T_1.val * F.val$$

The error occurs when the semantic action for $+$ is performed before $T.val$ for $b*c$ has been calculated.

For $a+b*c$, $b*c$ must be evaluated first and then $a+b*c$ is evaluated.

Program:

```
#include <stdio.h>
```

```
int main( )
```

```
{
```

```
    int a, b, c;
```

```
    int mul, result;
```

```
    printf("Enter the value of a:");
```

```
    scanf("%d", &a);
```

```
    printf("Enter the value of b:");
```

```
    scanf("%d", &b);
```

```
    printf("Enter the value of c:");
```

```

scanf ("-1-d", &c);
mul = b * c;
printf (" \n b*c = -1-d \n", mul);
result = a + mul;
printf (" a+(b*c) = -1-d \n", result);
printf (" Final result = -1-d \n", result);
return 0;
}

```

Output :

Enter value of a : 10

Enter value of b : 5

Enter value of c : 2

$b * c = 10$

$a + (b * c) = 20$

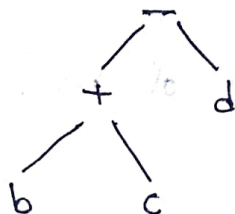
Final result = 20

2.

Error:

The faulty semantic action incorrectly attaches the + node directly under the - node.

Incorrect tree:



Here, two things are gone wrong:

→ The relationship $a*(b-c)$ is lost.

→ The $+$ node is incorrectly treated as direct child of $-$.

Program:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
struct node
```

```
{  
    char value;  
    struct node *left;  
    struct node *right;  
};
```

```
struct node* createNode(char value)
```

```
{  
    struct node* newNode;  
    newNode = (struct node*) malloc(sizeof(struct  
                                         node));  
    newNode->value = value;  
    newNode->left = null;  
    newNode->right = null;  
    return newNode;  
}
```

```
void printTree(struct node* root, int space)
```

```
{  
    int i;  
    if (root == null)  
        return;
```

```
printtree (root → right, space + 5);
```

```
for (i = 0; i < space; i++)
```

```
    printf(" ");
```

```
printf("-%c\n", root → value);
```

```
printtree (root → left, space + 5);
```

```
}
```

```
int main ( )
```

```
{
```

```
    struct node *a, *b, *c, *d;
```

```
    struct node *plus, *multiply, *minus;
```

```
    a = createnode ('a');
```

```
    b = createnode ('b');
```

```
    c = createnode ('c');
```

```
    d = createnode ('d');
```

```
    plus = createnode ('+');
```

```
    plus → left = b;
```

```
    plus → right = c;
```

```
    multiply = createnode ('*');
```

```
    multiply → left = a;
```

```
    multiply → right = plus;
```

```
    minus = createnode ('-');
```

```
    minus → left = multiply;
```

```
    minus → right = d;
```

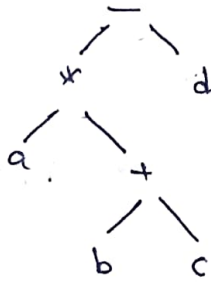
```
    printf ("Correct syntax tree:\n");
```

```
    printtree (minus, 0);
```

```
    return 0;
```

```
}
```

output :



3. The error occurs when the semantic action for multiplication

$$T \rightarrow T * F \quad T\text{-val} = T1\text{-val} * F\text{-val}$$

is executed before the addition:

$$E \rightarrow E_1 + T \quad E\text{-val} = E1\text{-val} + T\text{-val}$$

For $x = (a+b) * c$

the parser must find first $a+b$.

only after that can it calculate $(a+b) * c$.

Program :

```
#include <stdio.h>
```

```
int main ( )
```

```
{
```

```
    int a, b, c, add, mul, x;
```

```
    printf ("Enter value of a: ");
```

```
    scanf ("%d", &a);
```

```
    printf ("Enter value of b: ");
```

```
    scanf ("%d", &b);
```

```
    printf ("Enter value of c: ");
```

```
    scanf ("%d", &c);
```



```

add = a + b;
printf("a+b = %d\n", add);
mul = add * c;
printf("(a+b) * c = %d\n", add, c, mul);
x = mul;
printf("x = %d\n", x);
return 0;
}

```

Output :

Enter value of a: 10

Enter value of b: 20

Enter value of c: 3

$a + b = 30$

$(30) * 3 = 90$

$x = 90$

4. The grammar is:

$D \rightarrow T L$

$T \rightarrow \text{int}$

$L \rightarrow \text{id}, L$

$L \rightarrow \text{id}$

A faulty semantic rule might be:

$D \rightarrow T L \quad L\text{-type} = T\text{-type}$

$L \rightarrow \text{id}, L_1 \quad \text{id-type} = L\text{-type}$

The problem is that the type is assigned to the first id, but $L_1\text{-type}$ is never assigned.

So the propagation becomes:

$\text{int} \rightarrow a$

but stops there:

$\text{int} \rightarrow a \quad b \quad c$

Therefore, b and c do not receive the int type.

Program:

```
#include <stdio.h>
```

```
int main( )
```

```
{
```

```
    char a, b, c;
```

```
    char type[10] = "int";
```

```
    printf("Enter first identifier:");
```

```
    scanf("%-1-c", &a);
```

```
    printf("Enter second identifier:");
```

```
    scanf("%-1-c", &b);
```

```
    printf("Enter third identifier:");
```

```
    scanf("%-1-c", &c);
```

```
    printf("Type Propagation: \n");
```

```
    printf("%-1-c \rightarrow %-1-s \n", a, type);
```

```
    printf("%-1-c \rightarrow %-1-s \n", b, type);
```

```
    printf("%-1-c \rightarrow %-1-s \n", c, type);
```

```
    return 0;
```

```
}
```

Output:

Enter first identifier: a

Enter second identifier: b

Enter third identifier : c

Type Propagation:

$a \rightarrow \text{int}$

$b \rightarrow \text{int}$

$c \rightarrow \text{int}$

5. Error:

Given, expression: $a + b * c$

The error is in the semantic action for the $+$ production

A faulty translation scheme might use:

$E \rightarrow E + T$ $E.\text{place} = \text{newTemp}()$

$\text{emit}(E.\text{place} = E1.\text{place} + T.\text{place})$

while the grammar incorrectly treats the expression as:

$E \rightarrow E + \text{id}$

$E \rightarrow E * \text{id}$

This can cause parser to generate:

$t_1 = a + b$

$t_2 = t_1 * c$

The problem is that $+$ is being evaluated before the higher-precedence $*$ operation.

$a + b \rightarrow t_1 \rightarrow t_1 * c$

Program :

```
#include <stdio.h>

int main ( )
{
    int a, b, c, t1, t2;
    printf ("Enter the value of a:");
    scanf ("%i-%d", &a);
    printf ("Enter the value of b:");
    scanf ("%i-%d", &b);
    printf ("Enter the value of c:");
    scanf ("%i-%d", &c);
    printf ("Three address code = \n");
    t1 = b * c;
    t2 = a + t1;
    printf ("Intermediate results: \n");
    printf ("t1 = %i-%d \n", t1);
    printf ("t2 = %i-%d \n", t2);
    printf ("Final result = %i-%d \n", t2);
    return 0;
}
```

Output :

Enter the value of a: 10

Enter the value of b: 5

Enter the value of c: 2

Intermediate Results:

$$t_1 = 10$$

$$t_2 = 20$$

Final result = 20